



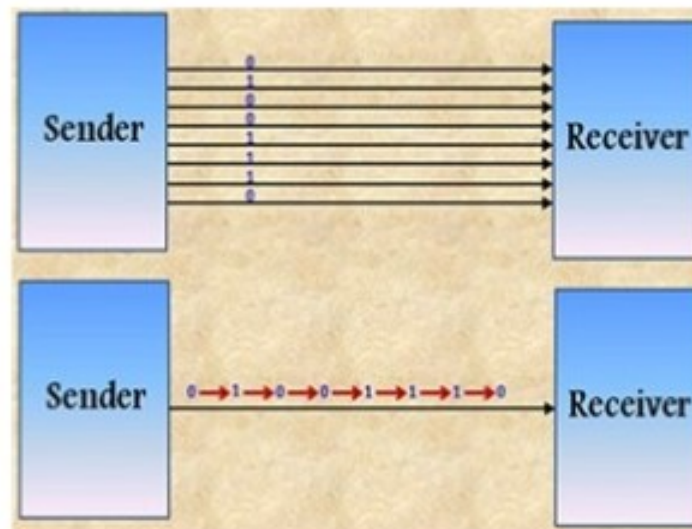
Unit 4

Embedded system and IOT Applications

-Dr.Ashish Nainwal
Assistant Professor
ECE Department
FET GK(DU) haridwar

Introduction

- We need communications to connect computers, microcontrollers, sensors, etc.
- There are two types of communications: parallel and serial.
- Parallel communications have higher transmission rates but requires more lines than serial communications.





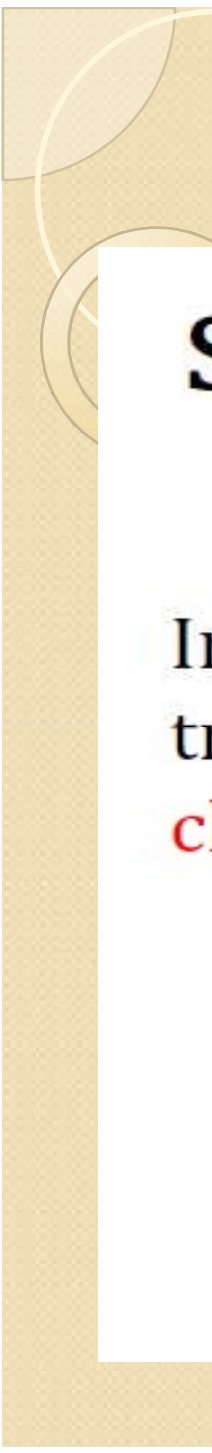
Modes of Connection

One-way: data sent in one direction only.

Half-duplex: two-way transfer, but only in one direction at a time.

Full-duplex: send and receive data simultaneously.

- RS-232 and SPI is full-duplex.
- I²C is half-duplex.



Synchronous vs. Asynchronous

In synchronous communication, both the transmitter and receiver share **a common clock. Thus, one line is assigned to the clock.**

- RS-232 (and also USB) is asynchronous
- SPI and I2C are synchronous.

Serial Protocols

- Recognize the following three serial interfaces:

- Standard RS-232 interface.
- SPI (Serial peripheral interface).
- I2C (Inter-integrated circuits interface).

} Very important in
embedded
system
interfacing.

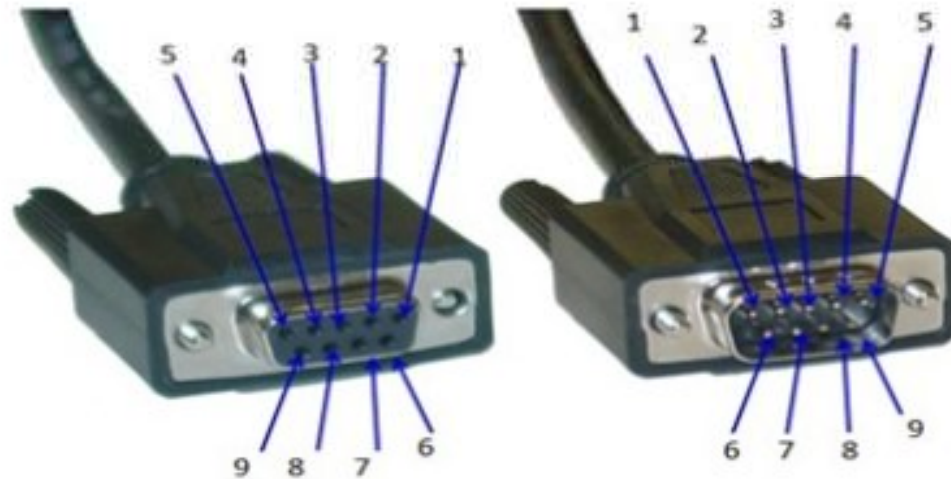


The RS 232

- RS-232: Recommended Standard 232
- Also called: “serial port” or COM port.
- Used to be a common interface to connect computer to other devices.
- Note: most modern laptops do not have serial port!

COM port

The DB-9 com port has 9 pins numbered as shown.

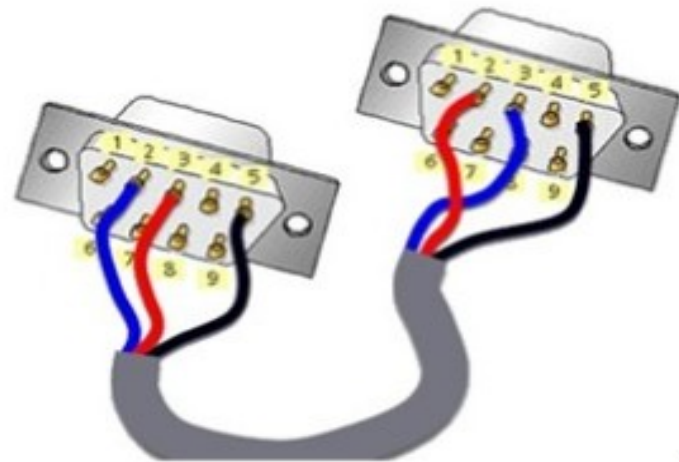
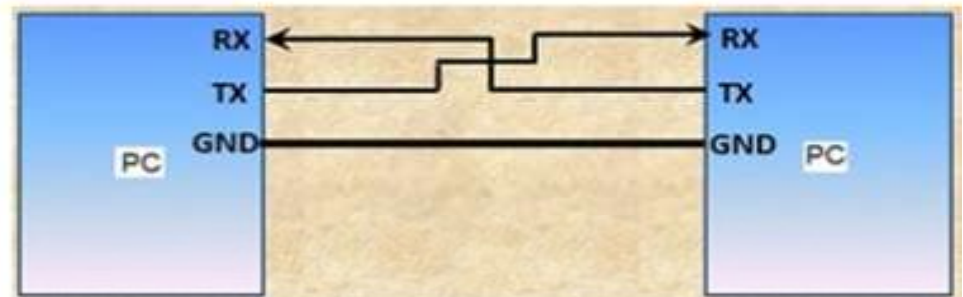


- Pin 2 – Tx (transmit data)
- Pin 3 – Rx (receive data)
- Pin 5 – Ground

The basic lines we need here

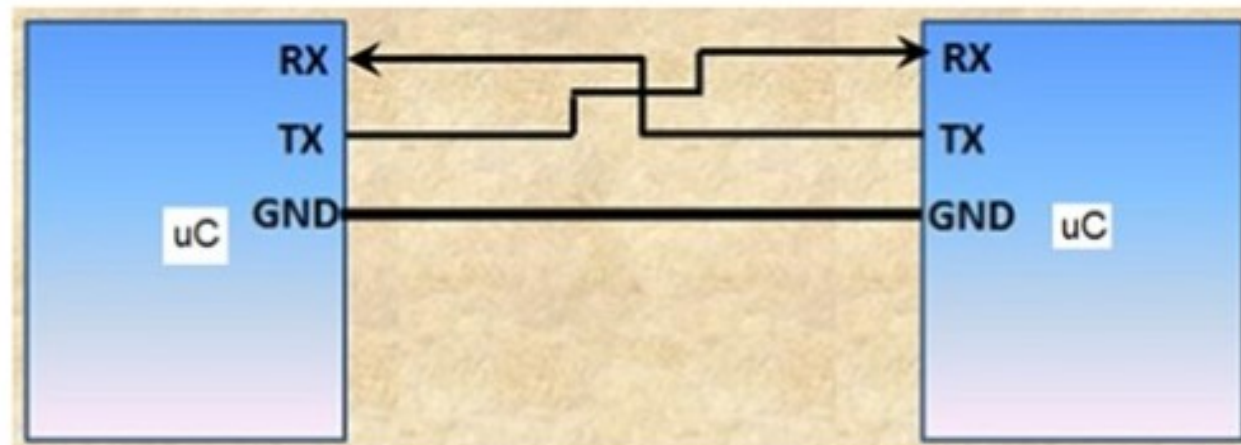
Connecting two PCs

- To connect two PCs, the simplest way is to connect
 - Pin 2 & Pin 3
 - Pin 3 & Pin 2
 - Pin 5 & Pin5
- This can be done with a cross cable.
- After that we should use a suitable software to implement the serial communications.



Connecting two microcontrollers (μC)

By the same way.



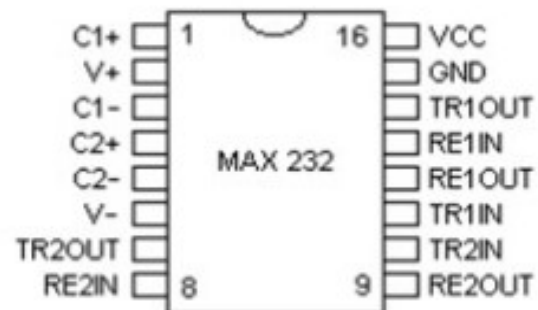
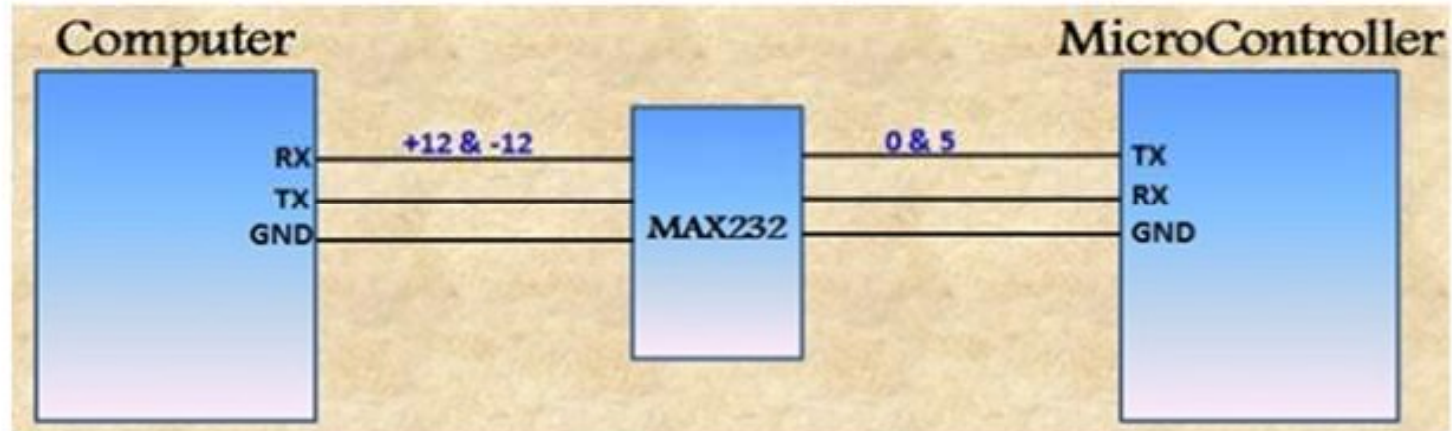
Connecting a PC to a μC

- This case is more difficult because The RS 232 logic levels are different from the (TTL 0&5V) levels.

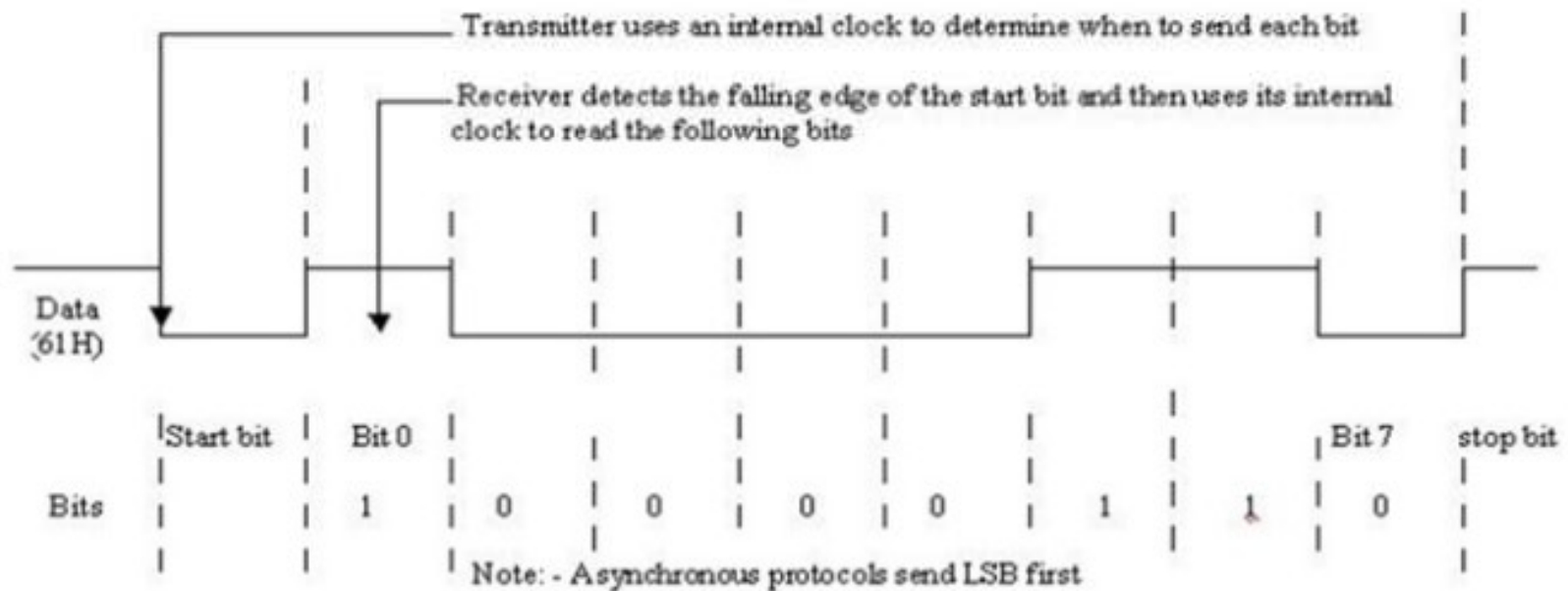
Logic Level	COM Port (RS232)	UART
On State	-3 To -25 Volt	Around 5 Volt
Off State	+3 To +25 Volt	Around 0 volt

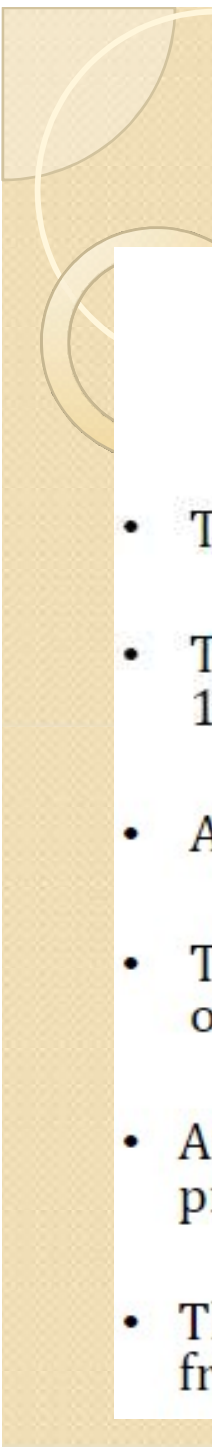
- The region between +3 and -3 volts is undefined.
- The serial port in the μC uses a unit called the UART which uses 0 and 5V logic levels.
- Due to differences in logic levels, we need to use a converter, e.g. MAX 232 IC.

Connecting a PC to a μ C



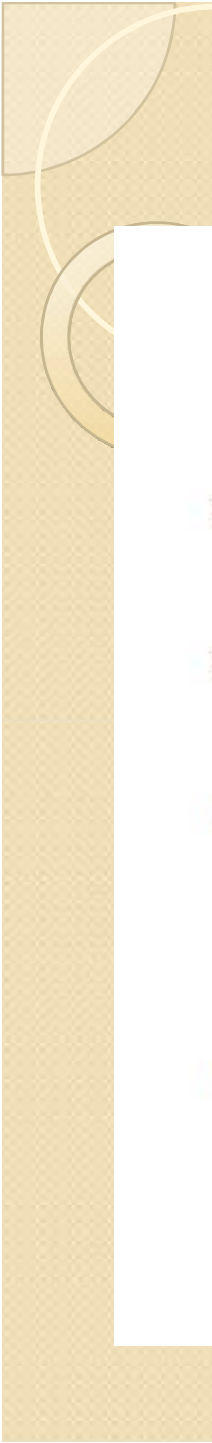
Serial data transfer protocol





Serial data transfer protocol

- The line is initially HIGH (no transmission).
- To start communication, the transmitter (Tx) pull the line low for 1 bit interval called the **start bit** .
- A number of **data bits** (e.g., 8 bits) are then sent in ASCII format.
- The Tx then send 1 bit HIGH (stop bit) to retain the original state of the line.
- After sending the character, an interrupt is issued to the processor.
- Thus, each transmitted character is packaged in a character frame.



A Character Frame Consists of:

- **Start bit:** the bit which signals the receiver that data is coming.
- **Data bits:** amount of actual data in a packet (5, 7, or 8 bits).
- **Parity:** It's used for error checking in serial communication. There are two types of parity: even and odd. The option of no parity is also available.
- **Stop bit:** is used to indicate the end of a single packet. Typical values are 1, 1.5, and 2 bits. The stop bits not only indicate the end of transmission but also give the computers some room for error in the clock speeds.

Tx and Rx must agree on:

1) The transmission rate (bit rate)

By adjusting the same bit rate in both transmitter and receiver, they can adjust the time of each bit and get the data correctly.

2) The number of data bits encoding a character

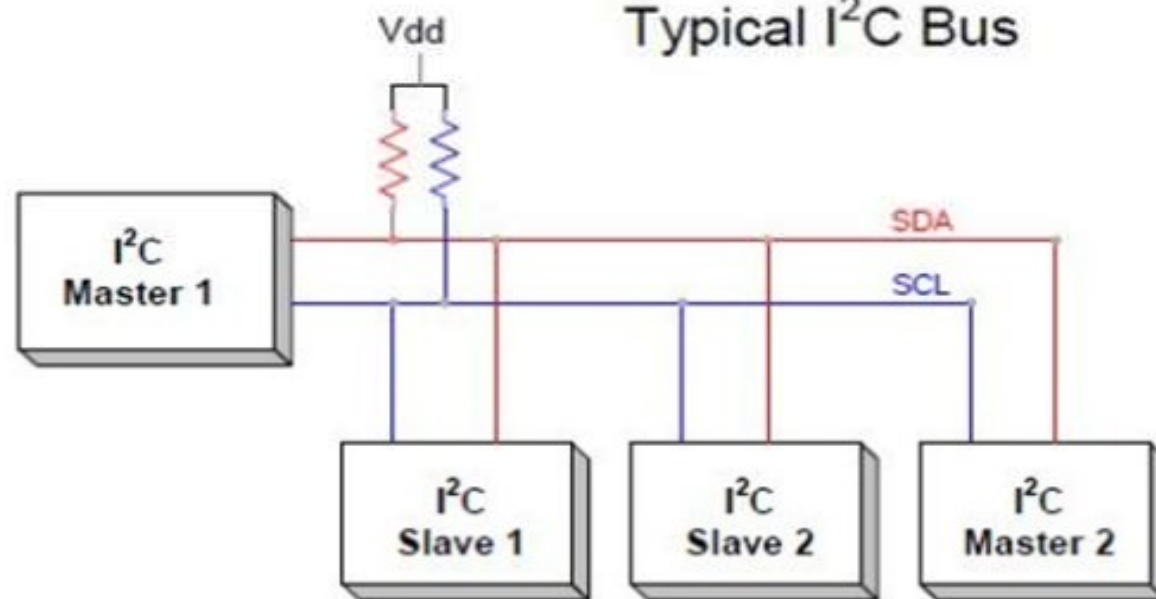
3) The optional parity bit

4) The number of stop bits

I²C Bus interface

I²C (Inter Integrated Circuits) bus is a bi-directional 2-wire bus widely used for serial communication between integrated circuits on the same circuit board and to attach low-speed peripheral devices, sensors, and components to embedded systems.

Typical I²C Bus





I²C Bus interface

- I²C has two bi-directional lines: Serial Data line (SDA) and Serial Clock line (SCL).
- I²C bus is a multi-master bus: any of the devices on the bus can initiate data transfer.
- The device that initiates communication is called MASTER, the device being addressed by the Master is called SLAVE.
- It is the master's responsibility to generate the clock.
- I²C has a 7-bit address space with 16 reserved addresses, which makes the maximum number of nodes 112.

How the master communicate with slave?

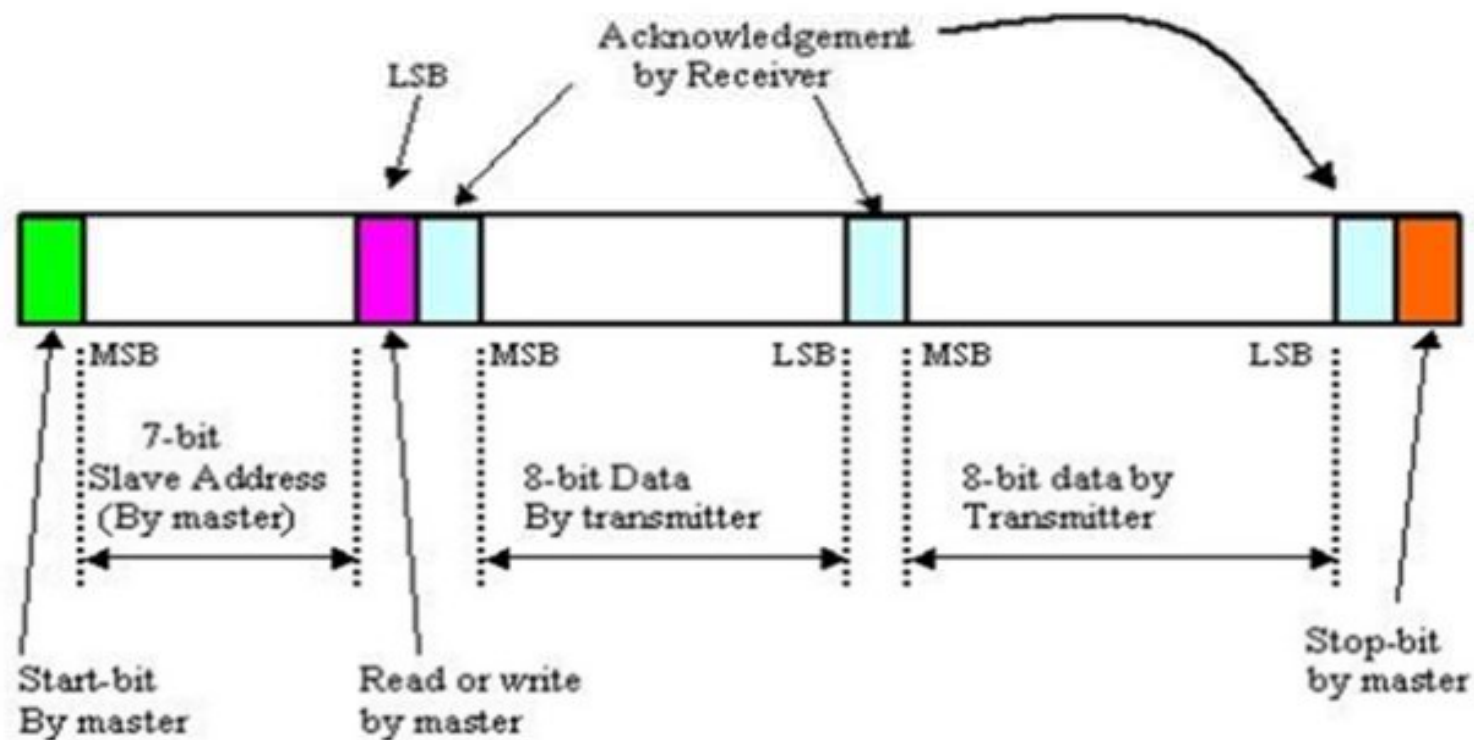
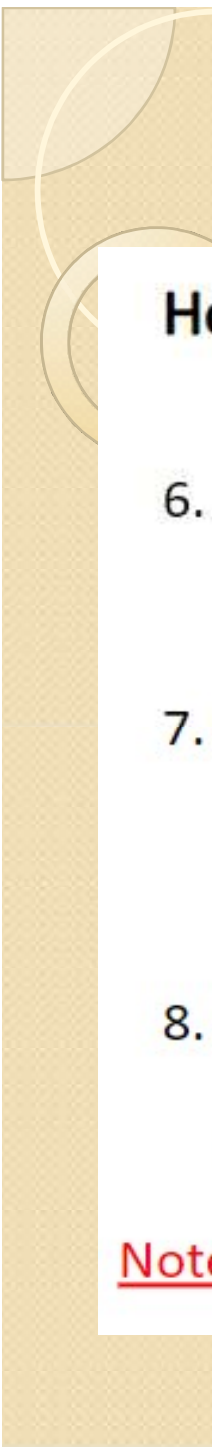


Fig-2 (I²C communication protocol)



How the master communicate with slave?

1. The MASTER waits until it sees *no activity* on the I²C bus (SDA and SCL are both HIGH).
2. The Master issues a start condition, informing other devices that it started to use the bus. The slave devices then start listening on the SDA line for instructions/data.
3. The Master Provides the clock signal on the SCL line.
4. The Master sends the unique binary address of the target device it wants to access.
5. Master puts a one-bit message on the bus telling whether it wants to SEND or RECEIVE data from slave.



How the master communicate with slave?

6. The Slave device with the matching address responds back with an acknowledgement signal.
7. The master then starts transmitting or receiving 8-bits of data to the receiver, which replies with a 1-bit acknowledgement and so on.
8. When the communication is complete, the master issues a stop condition indicating that everything is done. This action frees up the bus.

Note that only two devices exchange data during one 'conversation'.



I²C Advantage and disadvantage

Advantage: Only two wires are required. So, I²C is well suited for boards with many devices connected on the bus. This reduces cost & complexity of the circuit.

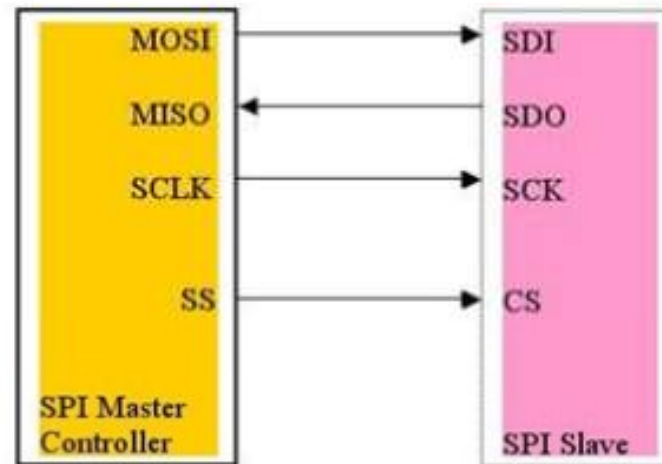
Disadvantage: However, due to the presence of only two wires, there is additional overhead of addressing and acknowledgments. This can be inefficient in simple configurations and a direct-link interface such as SPI might be preferred.



SPI Interface

- SPI is a 4-wire serial communications interface
- Used by many μp and μc to connect to devices such as ADC, memory modules, sensors, or other μp and μc .
- Speed up to 10Mbps. Due to its high speed, the bus lines cannot be too long, hence, the SPI is used only on the PCB.
- Full duplex.
- Synchronous type.

Data and control lines of the SPI and the basic connection:



An SPI protocol specifies 4 wires:

- Master Out Slave In (MOSI)
- Master In Slave Out (MISO)
- Serial Clock (SCLK or SCK) - *generated by the Master*
- Slave Select (SS) from master to Chip Select (CS) of slave

SPI protocol

- To begin communication, the bus master configures the clock, using a frequency supported by the slave device, typically up to a few MHz.
- The master select the desired slave by pulling the line (SS) low.
- Data transfer is organized by using Shift register in both transmitter and receiver.
- When there are no more data to be transmitted, the master stops its clock.



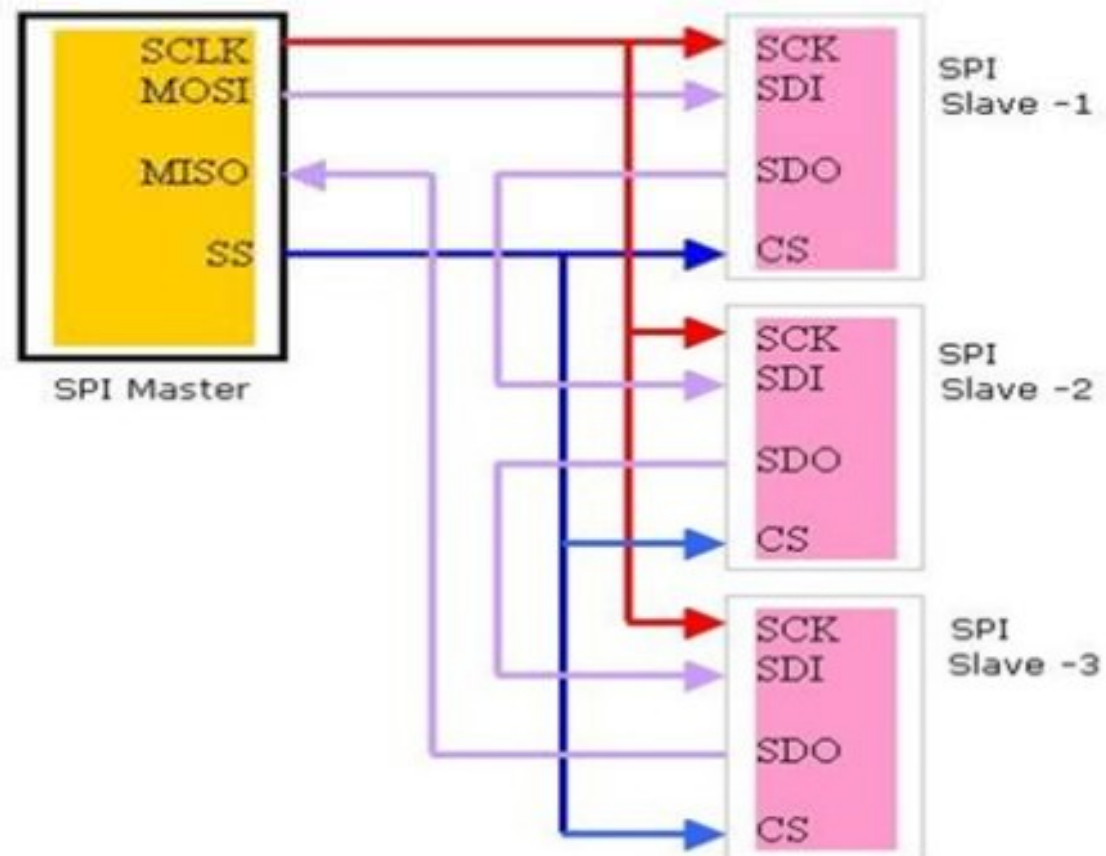
Configurations for Multiple Slaves

Suppose a master μc needs to talk to multiple SPI peripherals (slaves).

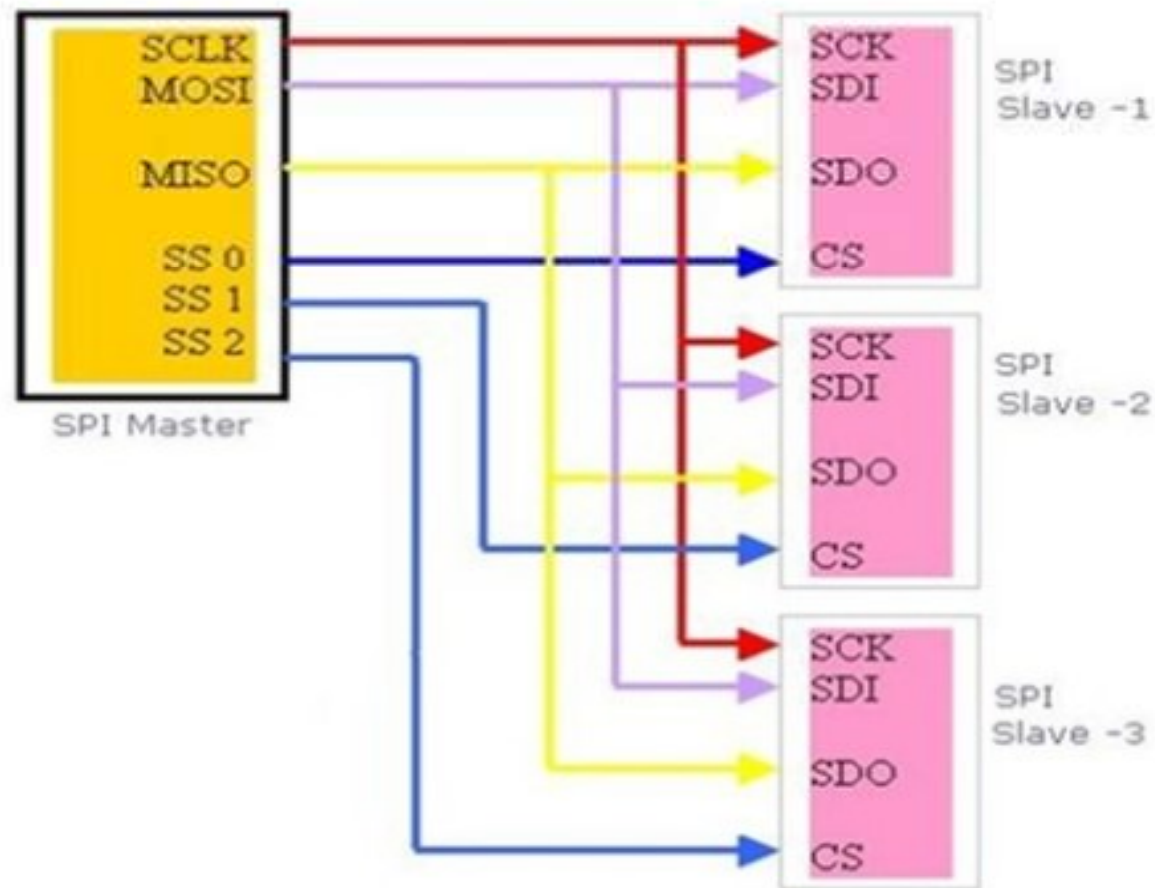
There are 2 ways to set things up:

1. Cascaded slaves or daisy-chained slaves
2. Independent slaves or parallel configuration

Daisy-chained (series) slave configuration



Independent (Parallel) Slave Configuration





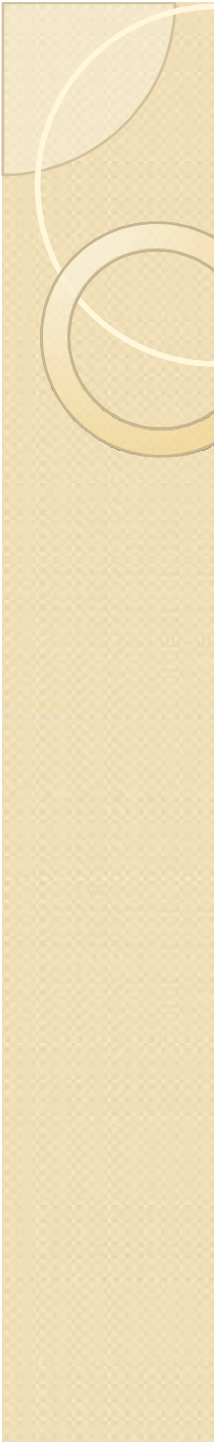
SPI advantages and disadvantages

Advantages:

1. Full duplex communication
2. Higher throughput than I²C protocol .
3. Slaves use the master's clock, and don't need precision oscillators.

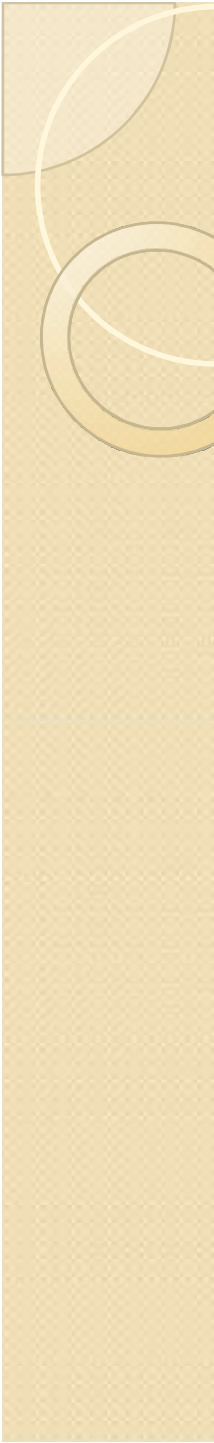
Disadvantage:

Requires more pins on IC packages than I²C.



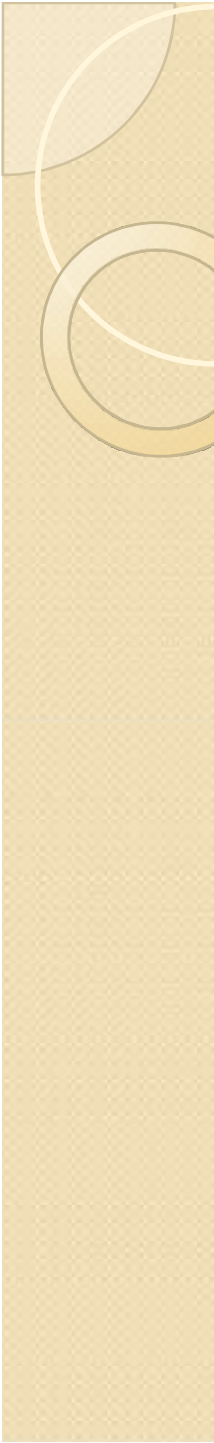
Parallel Communication Protocols

- Parallel communication transmits multiple bits simultaneously.
- Used in microprocessor data and address buses.
- Examples: Parallel Port, PCI.
- Advantages: High speed for short distances.
- Disadvantages: Crosstalk, synchronization issues.



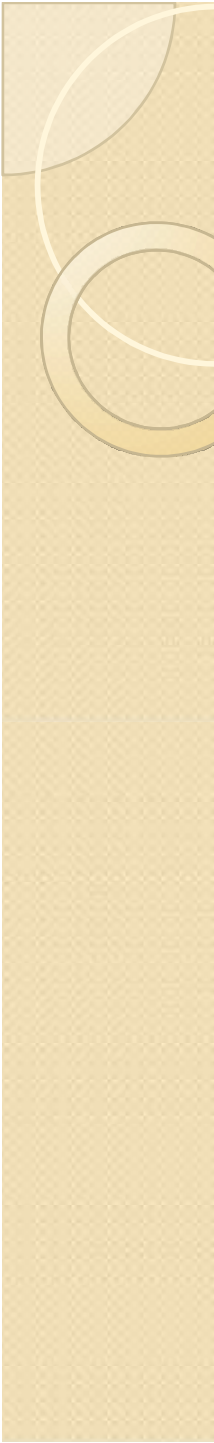
Wireless Communication Protocols

- Wireless communication uses radio waves.
- Wi-Fi: High speed, medium range.
- Bluetooth: Short range, low power.
- Zigbee: Low power, mesh networking.
- LoRa: Long range, low data rate.
- GSM: Cellular-based communication.

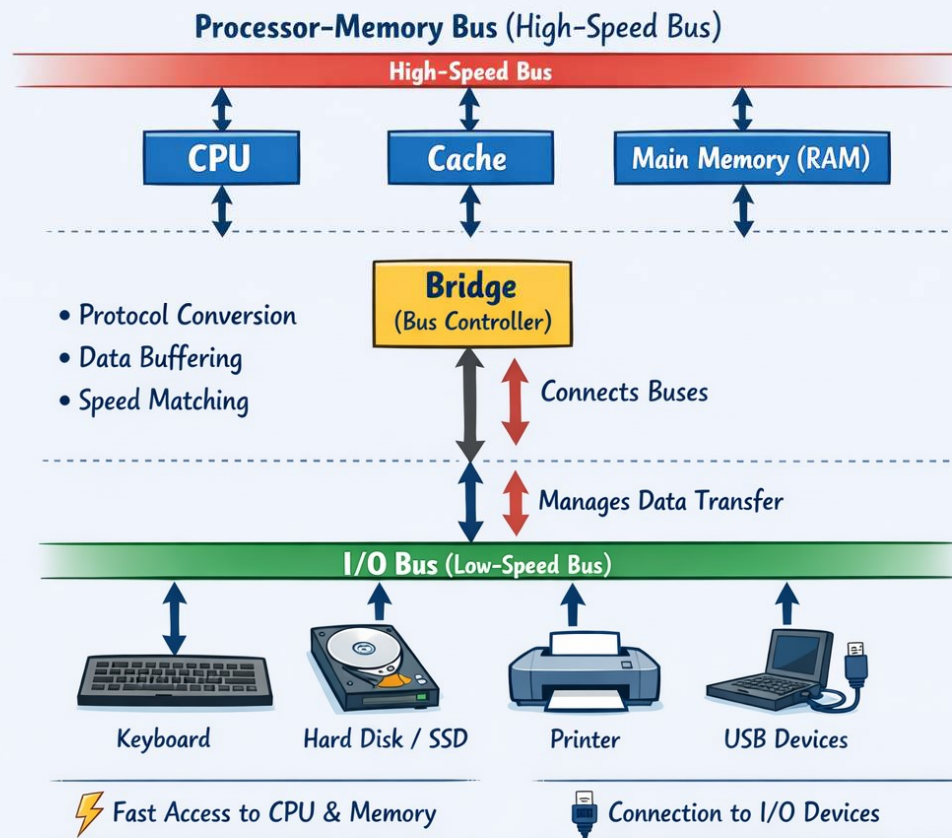


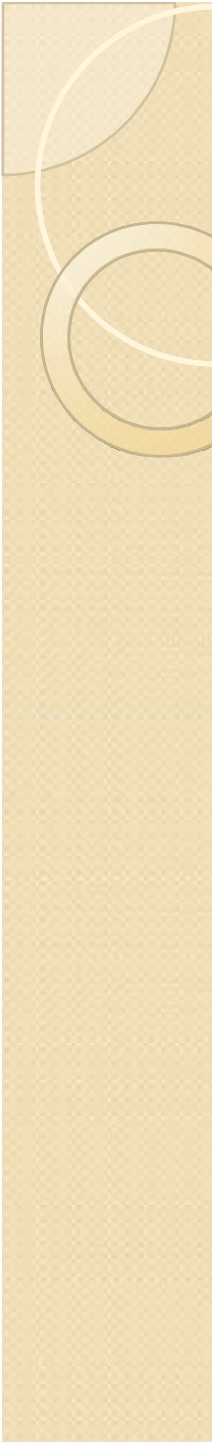
Multilevel Bus Architecture

- A **Multi-Level Bus Architecture** is a system design where **multiple buses are used at different hierarchy levels** instead of a single shared bus.
- This improves performance, reduces congestion, and allows faster communication between components.

- 
- Instead of one bus connecting everything:
 - System is divided into **levels (layers) of buses**
 - Each level connects specific components
 - Communication between levels happens through **bridges**

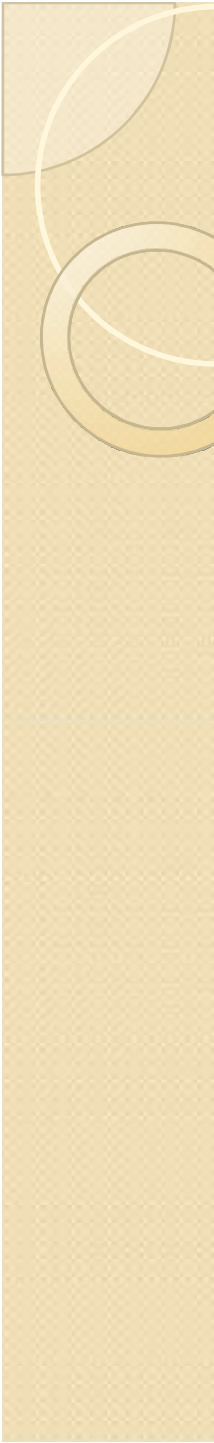
Multi-Level Bus Architecture





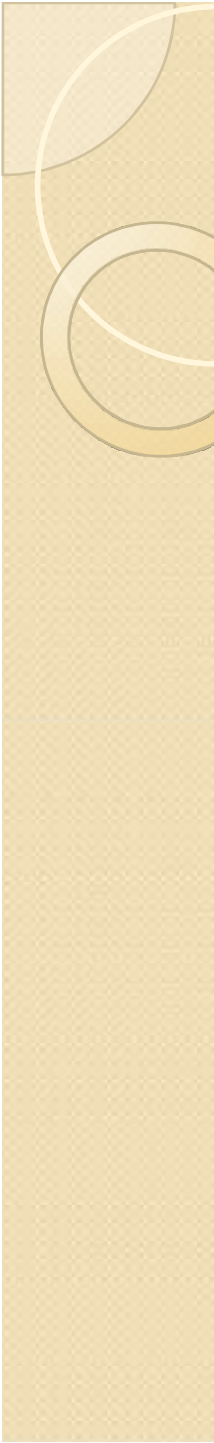
I. Processor-Memory Bus (High-Speed Bus)

- Connects:
 - CPU
 - Cache
 - Main Memory (RAM)
- Very fast (critical operations)
- Example: Front Side Bus (FSB), Memory Bus



2. I/O Bus (Lower-Speed Bus)

- Connects:
 - Hard disk
 - Keyboard
 - Printer
 - USB devices
- Slower than processor bus
- Example: PCI, USB

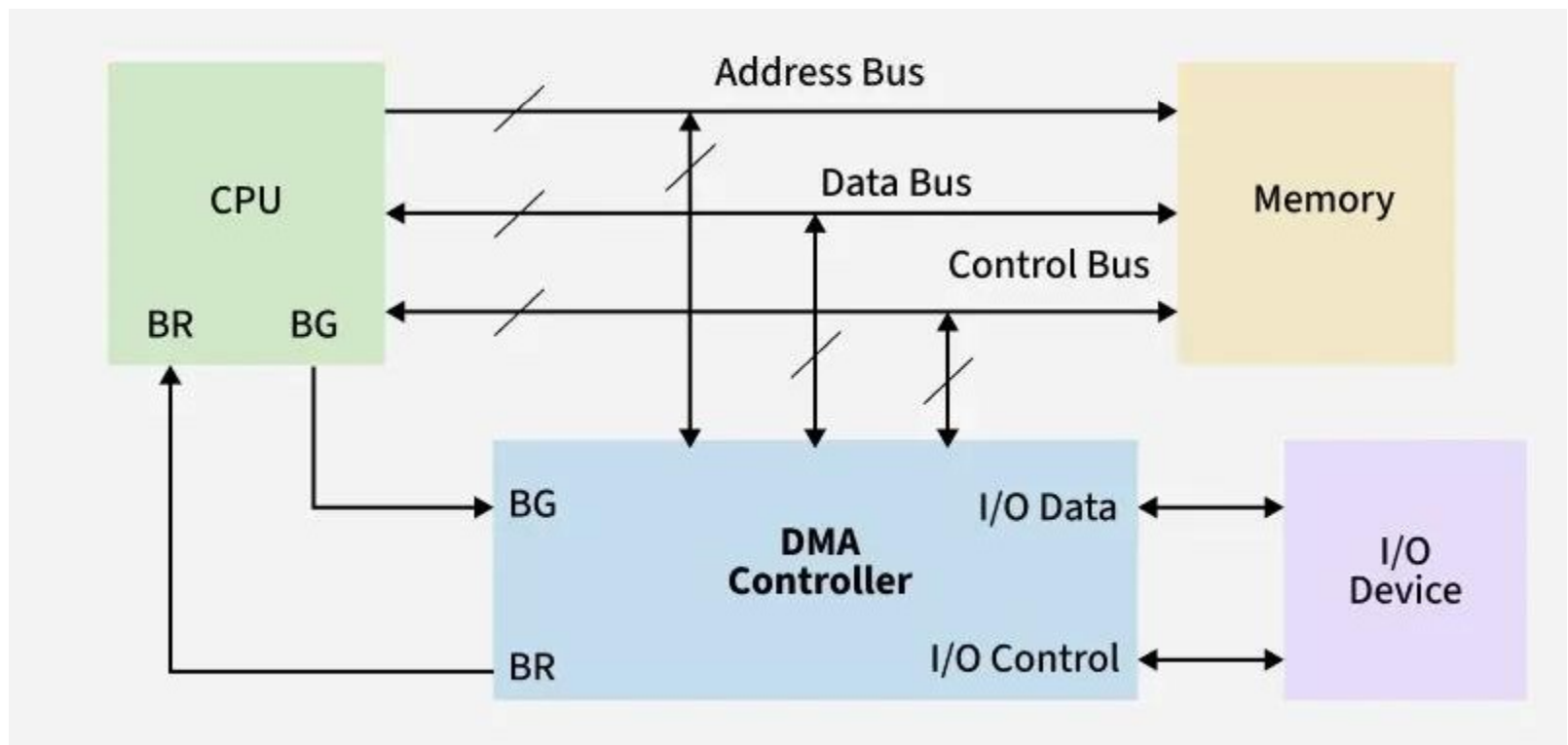


3. Bridge (Bus Controller)

- Connects different buses
- Handles:
 - Data transfer
 - Speed matching
 - Protocol conversion

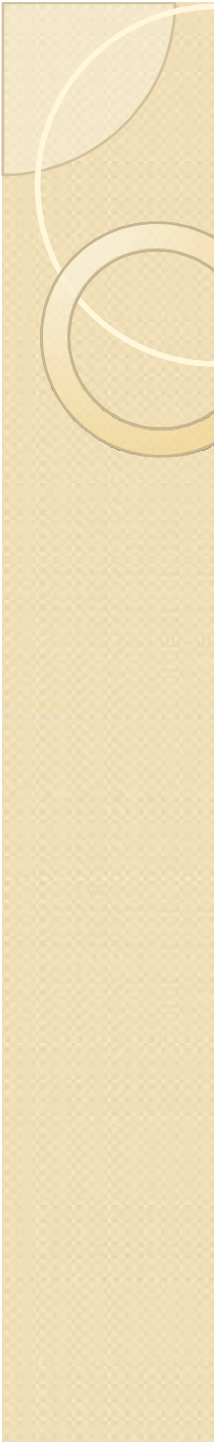
Direct Memory Access

- **DMA (Direct Memory Access)** is a technique that allows an I/O device to **transfer data directly to/from main memory without continuous CPU involvement.**



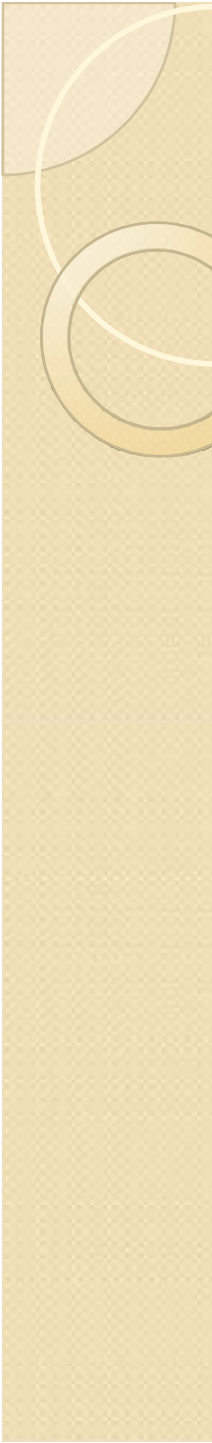
How DMA Works ?

- CPU gives:
 - Memory address
 - Number of bytes
 - Direction (Read/Write)
- DMA requests bus (**Bus Request - BR**)
- CPU gives permission (**Bus Grant - BG**)
- DMA transfers data **directly between I/O and memory**
- On completion → DMA sends **Interrupt** to CPU



Modes of DMA Transfer

- **1. Burst Mode (Block Mode)**
 - Transfers entire data in one go
 - CPU is stopped temporarily
 - Fast but CPU waits
- **2. Cycle Stealing Mode**
 - DMA transfers one byte at a time
 - CPU and DMA share bus
 - Slower but CPU keeps working
- **3. Transparent Mode**
 - DMA works only when CPU is idle
 - No CPU interruption
 - Slowest but efficient usage

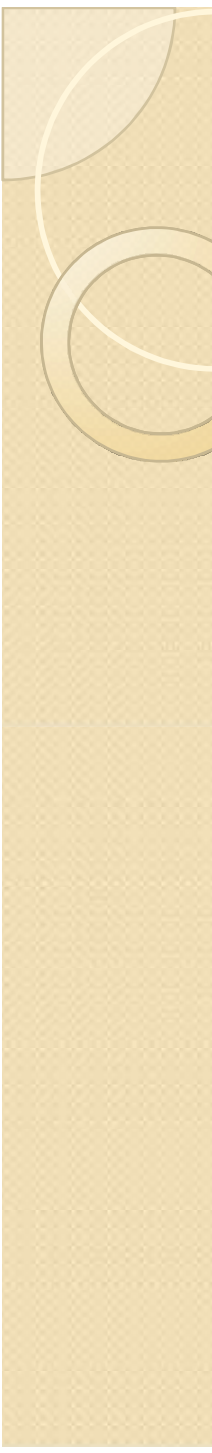


Advantages

- Faster data transfer
- Reduces CPU workload
- Efficient for large data (disk, network, etc.)

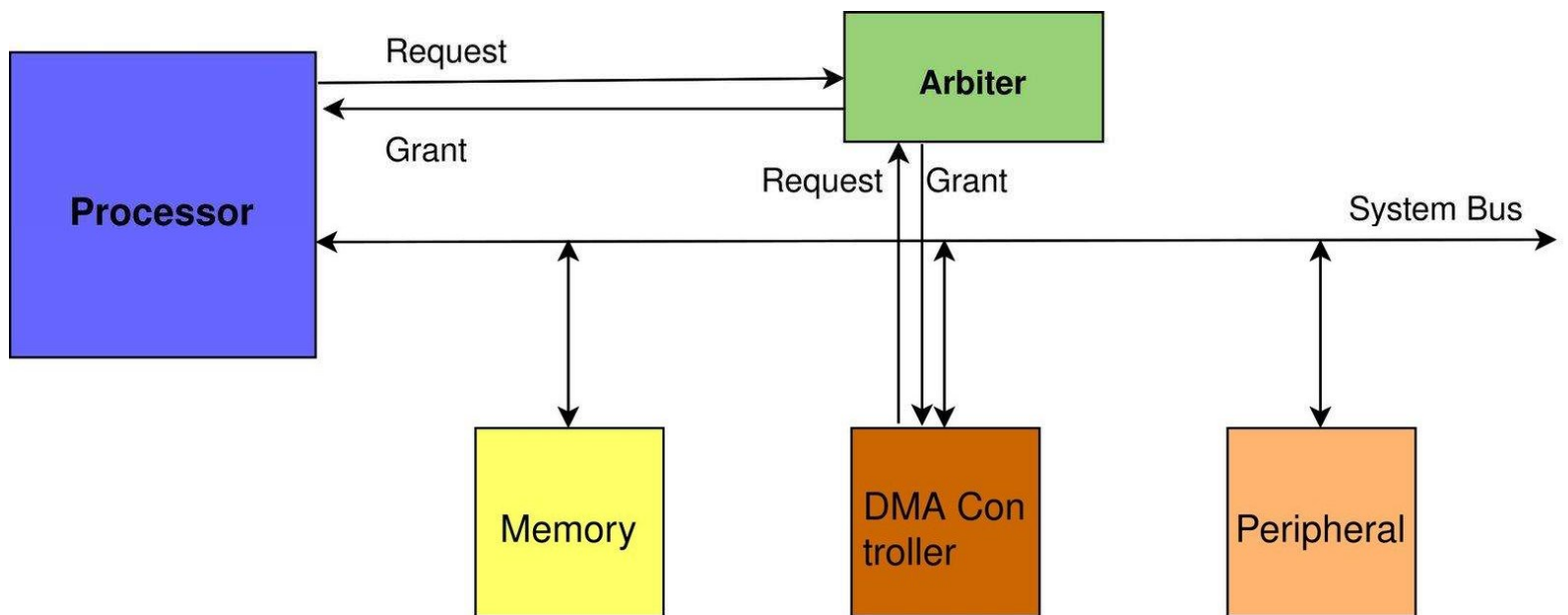
Disadvantages

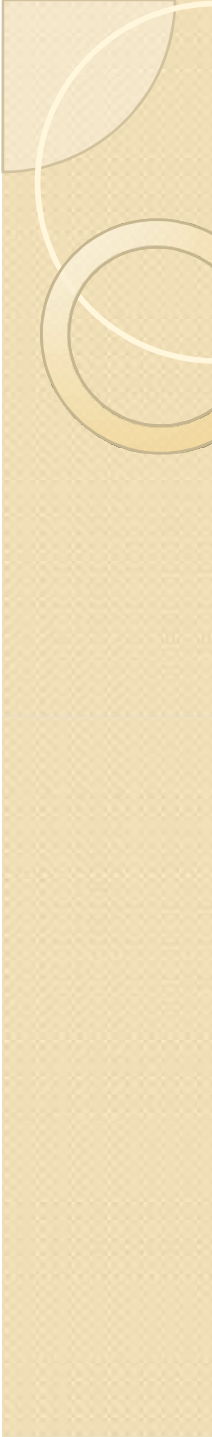
- Hardware complexity (DMA controller needed)
- Bus contention with CPU
- Debugging becomes difficult

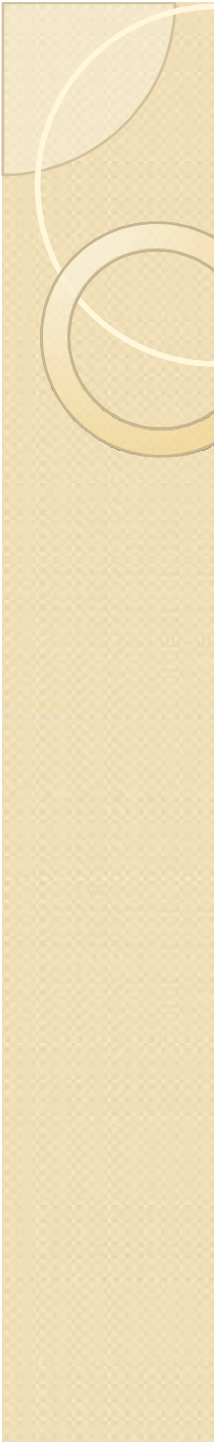


Arbitration

- **Arbitration** is the process of **deciding which device gets control of the system bus** when multiple devices request it at the same time.
- “Arbitration = Who gets the bus first?”

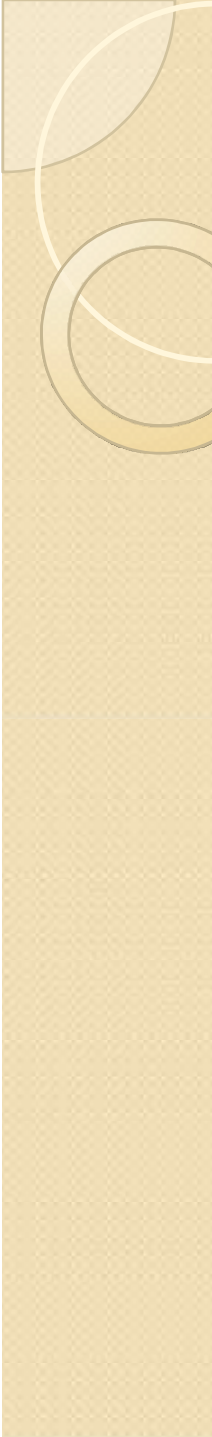


- 
- Multiple devices send a **Bus Request (BR)**
 - Arbiter checks priority
 - Grants access using **Bus Grant (BG)**
Selected device uses the bus
 - After completion → bus released



Types of Arbitration

- **I. Centralized Arbitration**
- Single controller (arbiter) decides
- Faster and simple
- **Methods:**
- **Daisy Chaining**
 - Devices connected in series
 - Fixed priority (nearest gets highest priority)
- **Parallel Arbitration**
 - Uses priority encoder
 - Faster but complex

- 
- **2. Distributed Arbitration**
 - No central controller
 - Devices decide among themselves
 - Uses self-selection logic

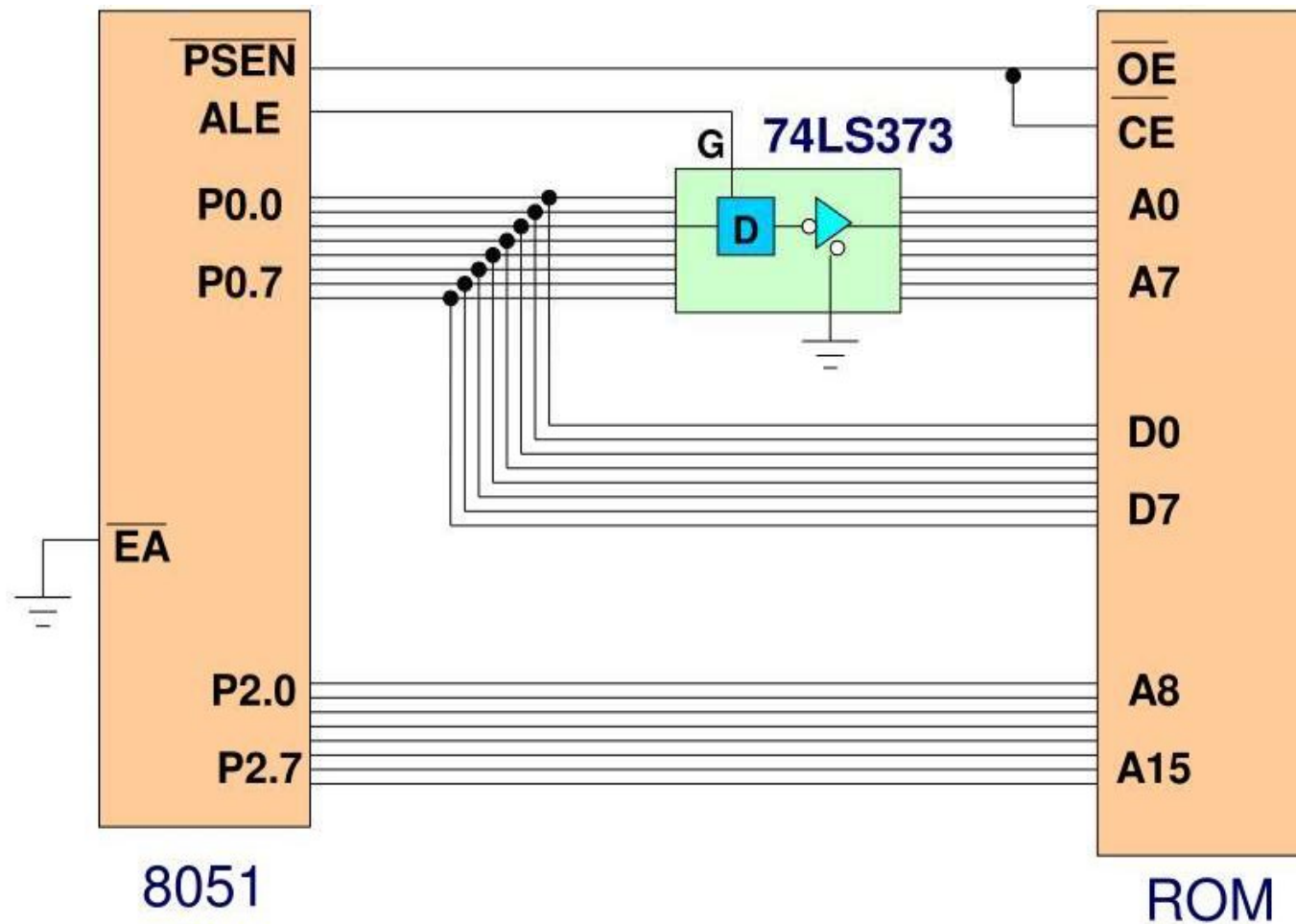
- **Advantages**

- Prevents bus conflicts
- Ensures proper data transfer
- Improves system efficiency

- **Disadvantages**

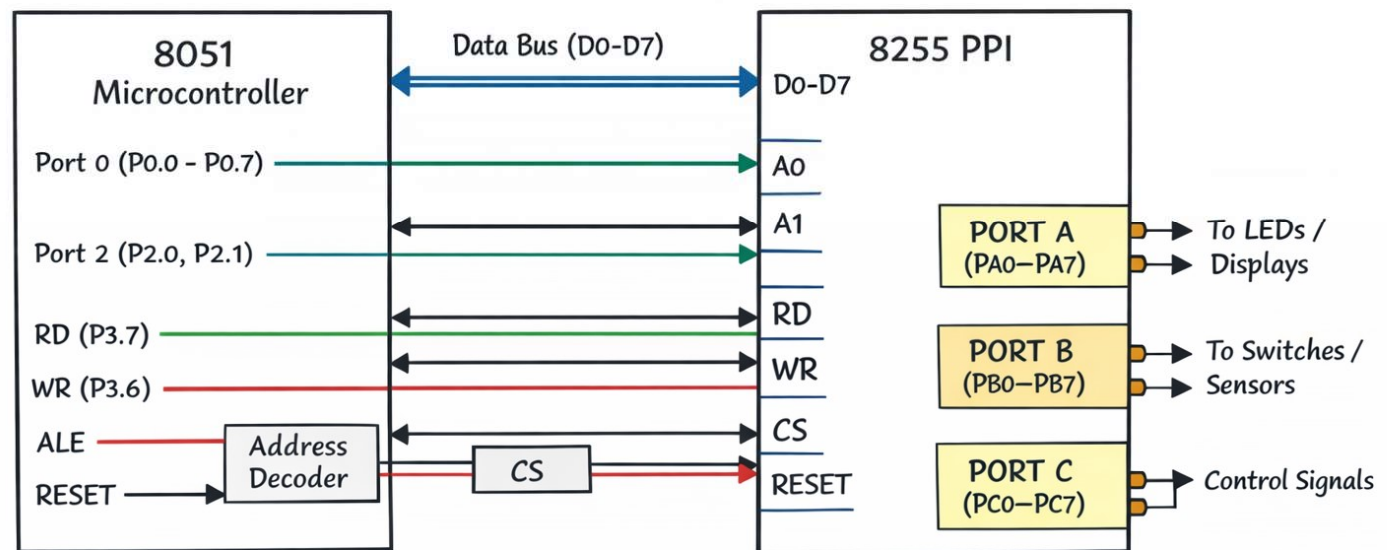
- Adds hardware complexity
- May cause delay if many requests
- Fixed priority can cause starvation

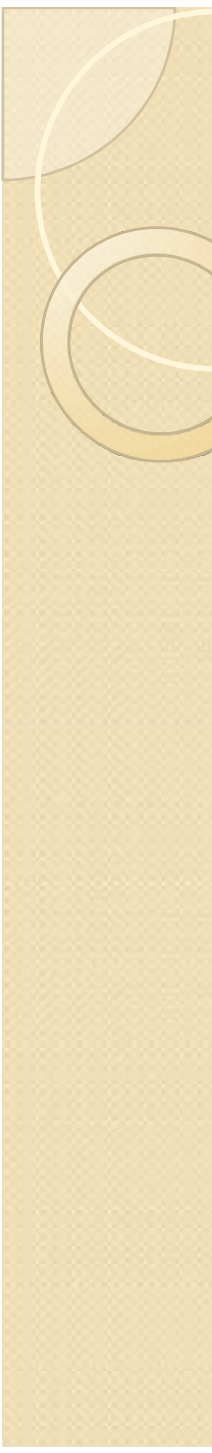
Latch Interconnection

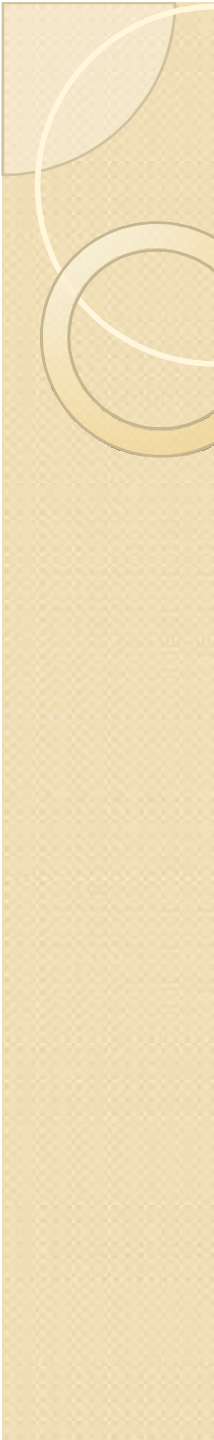


Interfacing of PPI

Interfacing 8255 PPI with 8051



- 
- `#include <reg51.h>`
 - `#define PORTA (*((unsigned char xdata *)0x8000))`
 - `#define PORTB (*((unsigned char xdata *)0x8001))`
 - `#define PORTC (*((unsigned char xdata *)0x8002))`
 - `#define CONTROL (*((unsigned char xdata *)0x8003))`
 - `void delay()`
 - `{`
 - `int i,j;`
 - `for(i=0;i<100;i++)`
 - `for(j=0;j<1000;j++);`
 - `}`



- `void main()`
- `{`
- `unsigned char data_in;`
- `// Initialize 8255`
- `CONTROL = 0x82;`
- `// 1000 0010`
- `// Port A → Output`
- `// Port B → Input`
- `// Port C → Output`
- `// Mode 0`
-
- `while(1)`
- `{`
- `data_in = PORTB; // Read switches from Port B`
- `PORTA = data_in; // Send same data to LEDs on Port A`
- `delay();`
- `}`
- `}`